CLASE 4 · TRACK DE IA

RAG

Retrieval-Augmented Generation

Cómo darle a un modelo información que no tiene

Recuperar → Aumentar → Generar

AGENDA

Objetivos de la clase

1. **Entender qué es RAG y por qué existe**
   El problema real que resuelve frente a un LLM solo.

2. **Distinguir las dos fases: recuperación y generación**
   Qué hace cada una y dónde se rompe cada una.

3. **Comprender el chunking**
   Por qué partimos los documentos y cómo afecta la búsqueda.

4. **Entender qué es un embedding**
   Texto convertido en vectores: la base de la búsqueda semántica.

01

El problema y la idea

Por qué un modelo, por sí solo, no alcanza

EL PROBLEMA

El LLM por sí solo no basta

1 Alucina

Cuando no sabe, igual responde. Genera texto plausible en vez de admitir el vacío.

2 Conocimiento congelado

Solo sabe lo que había en su entrenamiento. Nada posterior a esa fecha existe para él.

3 No conoce tus datos

Tu manual interno, tus documentos, tu base de clientes: nunca los vio.

4 No cita fuentes

No puede mostrar de dónde salió la respuesta, así que no es verificable.

DEFINICIÓN

¿Qué es RAG?

Arquitectura que busca información relevante en una fuente externa y se la entrega al modelo como contexto antes de que responda.

RETRIEVAL

Recuperación

Busca en la base de conocimiento los fragmentos que tienen que ver con la pregunta.

No usa el modelo. Es búsqueda.

GENERATION

Generación

El LLM recibe la pregunta más los fragmentos recuperados y redacta la respuesta.

Usa el modelo, pero acotado al contexto.

FLUJO

El pipeline de un RAG

1

Pregunta

El usuario escribe su consulta.

2

Búsqueda

Se buscan fragmentos relevantes en la base.

3

Contexto

Los fragmentos encontrados se seleccionan.

4

Prompt

Pregunta + contexto se arman en un solo prompt.

5

Respuesta

El LLM responde basándose en ese contexto.

Los pasos 2 y 3 son el retrieval. El paso 5 es la generación. Todo lo demás es tu aplicación.

02

Recuperar: primer intento

Búsqueda por palabras clave y sus límites

RETRIEVAL v1

Búsqueda por palabras clave

La forma más simple de recuperar: comparar las palabras de la pregunta con las palabras de cada documento. Si coinciden, el documento se considera relevante.

A favor

- Fácil de implementar y de explicar
- Rápida, sin costo de API
- Resultado predecible y auditable

En contra

- Compara letras, no significado
- No entiende sinónimos ni contexto
- Palabras comunes generan falsos positivos

LÍMITES

Dónde se rompe este enfoque

1 Sinónimos

Pregunto por 'auto' y el documento dice 'vehículo'. Para el buscador no tienen nada que ver.

2 Palabras vacías

Palabras como 'de' o 'qué' coinciden con todo y arrastran documentos irrelevantes.

3 Sin significado

Coincidir la palabra no es lo mismo que responder la pregunta.

Necesitamos buscar por significado, no por coincidencia de texto. Eso exige dos cosas: dividir bien los documentos y representarlos como números.

03

Chunking

Dividir el documento antes de buscar en él

CONCEPTO

¿Por qué dividir el documento?

1

Un documento largo mezcla muchas ideas

Si lo tratas como una sola unidad, su representación queda difusa: no es de un tema, es de diez.

2

Un fragmento chico representa una idea

Al dividir, cada pedazo tiene un tema claro y la búsqueda puede apuntar al fragmento correcto.

3

El contexto del modelo es limitado

No puedes mandarle el documento completo en cada consulta: cuesta tokens y diluye la respuesta.

MÉTODOS

Cuatro formas de dividir el texto

1 Por palabras

Corta cada N palabras. Sirve para textos largos y parejos, pero puede partir una idea al medio.

2 Por oraciones

Agrupa oraciones completas. Mantiene la coherencia semántica: ideal para textos explicativos.

3 Por párrafos

Cada párrafo es un chunk. Respeta la estructura original, pero deja chunks de tamaños muy dispares.

4 Por caracteres

Corta cada N caracteres. Control exacto del tamaño, el menos respetuoso del sentido.

PARÁMETROS

Tamaño del chunk y solapamiento

Tamaño

`chunk_size`

Chunk muy chico: pierde contexto y el fragmento no se entiende solo. Chunk muy grande: vuelve a mezclar temas.

Solapamiento

`chunk_overlap`

Repetir el final de un chunk al inicio del siguiente, para que una idea cortada justo en el borde no se pierda.

Cómo se ve el solapamiento

texto repetido entre chunks vecinos



04

Embeddings

Convertir el significado en números

DEFINICIÓN

¿Qué es un embedding?

Una representación del texto como lista de números, donde la posición de esos números refleja el significado del texto.

1 Otro modelo

Lo genera un modelo de embeddings, distinto del que produce texto. Su única salida es el vector.

2 Dimensión fija

Cada texto genera un solo vector, y ese vector siempre tiene la misma cantidad de números, sea una palabra o un párrafo completo.

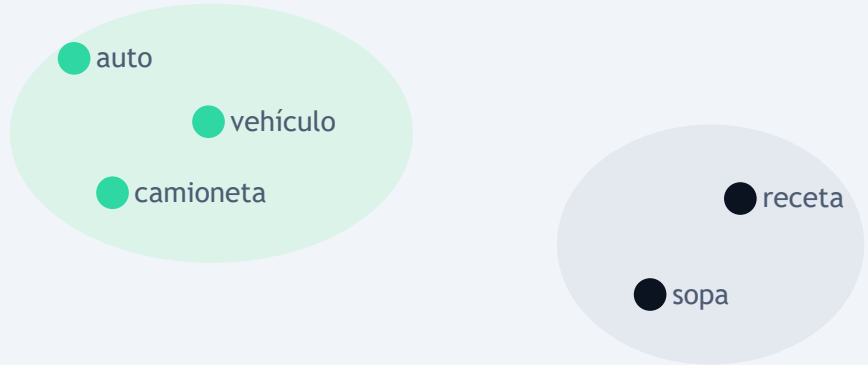
3 Significado, no palabras

Dos textos que dicen lo mismo con palabras distintas quedan cerca en el espacio.

INTUICIÓN

Vecinos en el espacio vectorial

Cada chunk queda ubicado en un espacio de muchas dimensiones. Los textos que hablan de lo mismo caen en zonas vecinas, aunque no compartan ni una palabra.



Espacio vectorial (representación simplificada)

Similitud coseno

Es la medida que usaremos para saber qué tan cerca están dos vectores. Se calcula entre la pregunta y cada chunk, y nos quedamos con los más cercanos.

Lo vemos la próxima clase.

RESUMEN

Los archivos de hoy

1. **`1-basic-rag.ipynb`**
   Un RAG por palabras clave: recuperar, armar el prompt y responder.

2. **`2-text-chunking.py`**
   Los cuatro métodos de división y el efecto del solapamiento.

3. **`3-embeddings-simple-rag.ipynb`**
   Cada chunk convertido en vector: la base de conocimiento.